

Flow-VLA-OPD: Paper Proposal

面向流匹配 VLA 模型的稠密在线策略蒸馏框架

OPD Papers 仓库·整理 by miracle-techlink

2026-05-15 ·v0.1 草案

Contents

1	执行摘要	3
2	1. Motivation	4
2.1	1.1 VLA 后训练的现状与困境	4
2.2	1.2 OPD 范式的吸引力	4
2.3	1.3 来自视频扩散的灵感	4
2.4	1.4 本项目的 Thesis	4
3	2. Related Work — 详细竞品对比	5
3.1	2.1 七个最直接相关工作的差异化	5
3.2	2.2 与 π _RL 的精细差异（最强直接竞品）	6
3.3	2.3 与 STARE-VLA 的精细差异	6
4	3. 核心思路：Flow-VLA-OPD	7
4.1	3.1 概览：三阶段流水线	7
4.2	3.2 三大设计要点	7
5	4. 算法完整伪代码	9
6	5. 实验设计	11
6.1	5.1 评估栈（来自用户决策）	11
6.2	5.2 Baseline 矩阵	11
6.3	5.3 核心实验问题（必答）	11
6.4	5.4 关键消融研究	12
6.5	5.5 真机补充实验（可选 Section 6）	12
7	6. 预期贡献 & 风险	13
7.1	6.1 预期贡献（写论文时的卖点）	13
7.2	6.2 主要风险与缓解	13
7.3	6.3 论文 framing 的 fallback 方案	13
8	7. 时间线与资源	15
8.1	7.1 假设资源	15
8.2	7.2 三个月时间线（目标 2026-08 arxiv 投稿）	15
8.3	7.3 关键决策点（gating questions）	15

9	8. 参考资料（按引用顺序）	16
9.1	直接相关论文	16
9.2	OPD 理论基础	16
9.3	VLA Backbone & Benchmark	16
9.4	已抓取本地解读	16

1 执行摘要

我们提出 **Flow-VLA-OPD**，第一个面向 **流匹配 (Flow Matching) / 扩散 (Diffusion)** 架构 **VLA 模型** 的稠密 On-Policy Distillation 后训练框架。核心方法是把 NVIDIA AnyFlow 在视频扩散中验证过的 **Flow Map Backward Simulation + DMD-style reverse-KL** 移植到机器人 VLA 场景，无需 **tokenize 动作**、无需可解的 **log-likelihood**、无需稀疏环境 **reward**。

关键差异化：相对于直接竞品 π _RL (2510.25889) 和 STARE-VLA (2512.05107)，Flow-VLA-OPD 用 **教师策略的稠密分布监督** 替代 sparse outcome reward，预期在 LIBERO / RoboTwin2.0 / SimplerEnv 上达成 **5-10 $\times$ sample efficiency**（参考 LLM 域 OPD 经验数据：Thinking Machines Lab, 2025）。

主要贡献预期：1. 首个在 flow-based VLA 上的 OPD 框架，证明 dense distribution-matching reward 可超越 sparse RL reward。2. 提出 **Action Chunk Backward Simulation (ACBS)** —— 把 AnyFlow 的 shortcut decomposition 适配到 4-8 步 VLA action chunk。3. 在 StarVLA-PI / StarVLA-GR00T 两个 backbone 上系统验证；3 个 benchmark 全面达到或超过 SOTA。4. 释放完整代码与配方，集成进 StarVLA codebase。

2 1. Motivation

2.1 1.1 VLA 后训练的现状与困境

主流 VLA 模型 (OpenVLA, π_0 , $\pi_{0.5}$, GR00T, RDT) 在预训练阶段拿到通用具身知识, 但在具体下游任务部署时需要 **后训练 (post-training)**。当前后训练范式有三条:

范式	采样	信号	代表工作	主要问题
Offline SFT	Off-policy	Dense	OpenVLA-OFT	Distribution shift, 灾难性遗忘
Online RL (PPO/-GRPO)	On-policy	Sparse outcome	π _RL, STARE-VLA, SimpleVLA-RL	sample inefficient, 高方差
Token-level OPD	On-policy	Dense	VLA-OPD (2603.26666)	必须 tokenize action, 不兼容 flow VLA

核心 gap: 业界主力 VLA 已转向 **流匹配架构** (π_0 / $\pi_{0.5}$ / GR00T 都是 flow matching action expert), 但所有现有 OPD 工作都要求离散 action token —— 无法直接应用。

2.2 1.2 OPD 范式的吸引力

Thinking Machines Lab (Oct 2025) 系统化提出的 OPD 在 LLM 上展示了惊人的效率: - 信息密度: dense 蒸馏 **$O(N)$ bits/episode** vs RL 的 **$O(1)$** —— 理论上 N 倍快 - 实测: Qwen3-8B 后训练比 RL 节省 **9-30 $\times$** 算力 - 抗遗忘: Reverse-KL 的 mode-seeking 性质天然保护预训练能力

VLA-OPD (Mar 2026) 已验证 OPD 思想在机器人 task 上工作 (LIBERO 48.9% $\rightarrow$ 87.4% 单条 demo), 但被 **token-level** 形式限制在 **OpenVLA-OFT** 类 **backbone**。

2.3 1.3 来自视频扩散的灵感

AnyFlow (NVIDIA, May 2026, arXiv 2605.13724) 把 OPD 成功迁移到 **任意步视频扩散**: - 不需要 log-likelihood —— 用 DMD-style score difference ($s_{\text{real}} - s_{\text{fake}}$) 近似 reverse-KL 梯度 - 不需要展开整条 ODE 链 —— Flow Map Backward Simulation 把 N 步压成 **3 个 shortcut** 段, 省 43-47% 显存 - 在 1.3B-14B 视频扩散模型上稳定 work

关键洞察: VLA 的 flow matching action expert 与视频扩散在数学上同构 (都是 PF-ODE + flow matching loss)。AnyFlow 的方法理论上可以直接迁移到 VLA 的 action chunk。

2.4 1.4 本项目的 Thesis

将 **AnyFlow** 风格的 **dense distribution-matching OPD** 移植到流匹配 VLA 模型, 用一个 **RL-fine-tuned** 同族强 teacher 在 student 自生成的环境轨迹上提供稠密监督, 实现免 tokenize、免 log-likelihood、免 sparse reward 的高效后训练。

3 2. Related Work — 详细竞品对比

3.1 2.1 七个最直接相关工作的差异化

#	工作	时间	架构	后训练范式	监督信号	是否需要 tokenize action	与本职工作差异
1	VLA-OPD [2603.26666]	2026-03	OpenVLA (discrete)	Token-level reverse-KL OPD	Dense token logits	$\times$ 必须	我们做 continuous 版本, 兼容 π_0 /GR00T π _RL 是 RL, 我们是 distillation; 我们提供 dense reward
2	π_RL [2510.25889]	2026-03	π_0 , $\pi_{0.5}$ (flow)	Online RL (Flow-Noise / Flow-SDE + PPO)	Sparse outcome reward + critic	$\times$	
3	STARE-VLA [2512.05107]	2025-12	OpenVLA 7B + $\pi_{0.5}$ base	Stage-shaped RL + DPO (IPI pipeline)	Stage-wise dense reward shaping	$\times$	
4	DiG-Flow [2512.01715]	2025-12	flow VLA	Discrepancy-guided representation regularization	训练时的几何正则项	$\times$	DiG-Flow 是预训练正则项, 与我们正交可叠加
5	ConRFT [2502.05450]	2025-02	Consistent policy VLA	BCy+ Q-learning + consistency policy	Q-value + intervention	$\times$	ConRFT 是 RL-based; 我们是 distillation-based
6	One-Step Flow Policy [2603.12480]	2026-03	Flow visuo-motor	Self-distillation (no teacher)	self-consistency + self-guided	$\times$	OFP 自蒸馏没外部知识注入; 我们注入 teacher 新能力
7	SDM Policy [2412.09265]	2024-12	Diffusion policy	Score+Distribution matching $\rightarrow$ 1 step		$\times$	SDM 只压缩推理步, 不提升能力; 我们做能力提升
★	Flow-VLA-OPD (本工作)	2026	π_0 / $\pi_{0.5}$ / GR00T (flow)	Dense distribution matching OPD	Score-based reverse-KL	$\times$	首个适用于 flow VLA 的 OPD

3.2 2.2 与 π _RL 的精细差异（最强直接竞品）

π _RL 的核心贡献是 让 **flow VLA 能做 RL**: - 困难: flow matching 的最终动作 log-likelihood 不可解析 - 解 1: **Flow-Noise** — 在 denoising 过程加 learnable noise network → 转为 discrete-time MDP 可精确算 log-likelihood - 解 2: **Flow-SDE** — 把 deterministic ODE 转为 SDE → 双层 MDP 用 PPO

我们 完全跳过 **log-likelihood** 问题: - DMD-style score difference $s_{\text{real}} - s_{\text{fake}}$ 是 reverse-KL 梯度的 score-based 估计, 根本不需要 **log-likelihood** - 不需要 critic / 价值函数 - 不需要 PPO 的 trust region 工程

维度	π _RL	Flow-VLA-OPD
信号来源	环境 sparse reward	Teacher 稠密分布
是否需要 log-likelihood	✓ 整篇核心问题	✗ 完全跳过
是否需要 critic	✓ (V_expert 或 V_vlm)	✗
是否需要 SDE/MDP 改造	✓	✗
信息密度 (理论)	O(1) per episode	O(N·T) per episode

3.3 2.3 与 STARE-VLA 的精细差异

STARE-VLA 用 **任务结构先验** (stage segmentation) 解决 long-horizon credit assignment: - 用 end-effector 几何阈值定义 stage 边界 - 每个 stage 一个 cost + dense potential reward - StA-TPO (offline preference) + StA-PPO (online)

我们不依赖任务结构: - Dense supervision 直接来自 teacher, 与任务无关 - 自动覆盖 “long-horizon” 问题 (teacher 已经会做整条轨迹)

维度	STARE-VLA	Flow-VLA-OPD
需要 stage 分段	✓	✗
任务通用性	限于 pick-and-place 类	任意操作任务
依赖外部 teacher	✗	✓ (但 teacher 来自现成 RL 训练)
后训练算法栈	DPO + PPO	OPD (单一目标)

4 3. 核心思路: Flow-VLA-OPD

4.1 3.1 概览: 三阶段流水线

仿照 VLA-OPD 的高层骨架, 但把 **token-level supervision** 全部替换成 **score-based distribution matching**。

Phase 1 (On-Policy Rollout):

Student π_θ 在环境中执行, 收集 trajectory $\tau = (o_1, A_1, \dots, o_T, A_T)$

其中 $A_t = [a_{\{t,0\}}, \dots, a_{\{t,H-1\}}]$ 是 H 步 action chunk

对每个 chunk: 从 noise $z_0^T \sim N(0, I)$ 经 K 步 Euler 解 ODE 得到 A_t

Phase 2 (Dense Teacher Score Labeling):

Teacher π_{tea} (frozen, 同 backbone 不同 checkpoint) 在 student 自生成的 observation 上计算:

- Teacher 的 velocity field $v_{tea}(A_t^T, o_t)$
- 在 student 经过的 noisy state z^τ 上的 fake score s_{fake} (从辅助 critic)

Phase 3 (Score-based Reverse-KL Update):

对每个 chunk, 对 K 步 ODE 用 Action Chunk Backward Simulation (ACBS):

- 采 inference budget N, 对应 chunk size K, $t = r + (K-r)/N$
- 三段 shortcut: $T \rightarrow t$, $t \rightarrow r$, $r \rightarrow 0$
- 在 z_0 上算 DMD gradient $\propto (s_{real} - s_{fake})$, 反传到 ACBS chain

4.2 3.2 三大设计要点

4.2.1 3.2.1 Action Chunk Backward Simulation (ACBS) — 关键工程创新

VLA 与视频扩散的不同: - 视频扩散 chunk 通常 4-32 steps (NFes) - VLA action chunk 通常 4-8 denoising steps (H=10-50 动作步)

直接展开 K 步 ODE 反传太贵。仿照 AnyFlow 的 Flow Map Backward Simulation:

Algorithm 1 (Action Chunk Backward Simulation):

Input: 初始噪声 $z^T \sim N(0, I)$, observation o_t , ODE 总步 K

Sample $s \sim \text{Uniform}(1, \dots, K)$ # inference budget

Sample $t \sim \text{Uniform}(1, \dots, s)$ # target gradient timestep

$r = t - K/s$ # shortcut next timestep

$z^t = f_\theta(z^T, o_t, T, t)$ # shortcut 1, with grad

$z^r = f_\theta(z^t, o_t, t, r)$ # target transition, with grad

$A_t = f_\theta(z^r, o_t, r, 0)$ # shortcut 2 = final action, with grad

return A_t # 仅 3 个 forward pass with grad, vs K 个

预期效益 (按 AnyFlow Tab.5 同比例外推): - K=4 步 chunk: 训练成本 +15% (vs 1-step naive backward) - K=8 步 chunk: 训练成本 **-30% vs full backward**

4.2.2 3.2.2 Score-based Reverse-KL Reward — 跳过 log-likelihood

仿照 AnyFlow 第 3.2 节, 但把 score model 改为 **VLA-aware**:

$$\nabla_{\theta} \mathcal{L}_{\text{Flow-VLA-OPD}} = -\mathbb{E}_{t,z,o} \left[(s_{\text{real}}(z^{\tau}, o, \tau) - s_{\text{fake}}(z^{\tau}, o, \tau)) \frac{\partial f_{\theta}(z, o)}{\partial \theta} \right]$$

- s_{real} 来自 frozen teacher (e.g., 用 SimpleVLA-RL 或 π_{RL} 训练过的 stronger expert)
- s_{fake} 是一个轻量学习的 score critic (DMD 标准做法), 与 student 同步更新

为什么这等价于 **reverse-KL**: - Reverse-KL gradient = $\mathbb{E}_{x \sim \pi_{\theta}} [\nabla_{\theta} \log(\pi_{\theta}/\pi_{\text{tea}})]$ - 在连续分布上, $\nabla \log p = \text{score}$, 所以差值 $s_{\text{real}} - s_{\text{fake}}$ 就是 reverse-KL gradient 的 score-based estimator - 此估计器在 Tianwei Yin et al. (DMD, CVPR 2024) 上已被证明 unbiased + 低方差

4.2.3 3.2.3 Teacher 的来源 — 工程上最关键的决定

候选方案:

方案	优点	缺点
A. SimpleVLA-RL trained π_0	直接可用, VLA-OPD 也用	单任务 expert
B. π_{RL} trained $\pi_0/\pi_{0.5}$	flow-native, 已发布 checkpoint	性能可能不如 A
C. 更大 size flow VLA (GR00T-N1 / $\pi_{0.6}$)	跨 size 蒸馏	资源贵, 可能动作空间不一致
D. Cosmos-Predict2 World Model	AnyFlow 同源	不是 action policy

主流程用 **A** (与 VLA-OPD 直接 head-to-head 公平对比), 在 ablation 章节做 B/C 对比。

5 4. 算法完整伪代码

Algorithm 2: Flow-VLA-OPD Training Loop

Input:

- Student policy π_θ (flow-based VLA), 来自 1-traj SFT 或 SFT-from-base
- Teacher policy π_{tea} (frozen, e.g., SimpleVLA-RL trained 同 backbone)
- Score critic s_{fake_ϕ} (initialized from student)
- Environment Env (LIBERO / RoboTwin / SimplerEnv 实例)
- Group size G (number of trajectories per prompt)
- Chunk size H, ODE steps K
- Optimizers: AdamW for θ and ϕ

while not converged do

————— Phase 1: Group On-Policy Rollout —————

for prompt o in batch do

for $i = 1, \dots, G$ do

Roll out $\tau_i = (o_0^i, A_0^i, \dots, o_T^i, A_T^i)$ using π_θ in Env
每个 A_t^i 是用 K 步 Euler 解 ODE 生成的 H-长 action chunk

end for

Add $(o, \{\tau_i\}_{i=1}^G)$ to buffer

end for

————— Phase 2: Compute ACBS-based DMD Gradient —————

for $(o, \{\tau_i\})$ in buffer do

for τ in $\{\tau_i\}$ do

for each chunk index t in trajectory do

ACBS: 3-段 shortcut 展开

Sample $s \sim U(1..K)$, $t_{inner} \sim U(1..s)$, $r_{inner} = t_{inner} - K/s$

$z^T = \text{randn_like}(A_t)$

$z^t = f_\theta(z^T, o_t^\tau, T, t_{inner})$ # with grad

$z^r = f_\theta(z^t, o_t^\tau, t_{inner}, r_{inner})$ # with grad

$A_t^{\text{pred}} = f_\theta(z^r, o_t^\tau, r_{inner}, \theta)$ # with grad

DMD-style reward

Sample $\tau' \sim U[0,1]$

$z'_\tau = (1 - \tau') * A_t^{\text{pred}} + \tau' * \text{randn_like}(A_t^{\text{pred}})$

$\text{score_real} = \text{compute_score}(\pi_{tea}, z'_\tau, o_t^\tau, \tau')$ # teacher score

$\text{score_fake} = s_{fake_\phi}(z'_\tau, o_t^\tau, \tau')$ # fake score

Gradient for student

$\text{grad}_\theta += \text{stop_gradient}(\text{score_real} - \text{score_fake}) * \partial A_t^{\text{pred}} / \partial \theta$

Update fake score critic (denoising loss)

$\text{loss}_\phi = ||s_{fake_\phi}(z'_\tau, o_t^\tau, \tau') - \text{true_velocity}(A_t^{\text{pred}}, z'_\tau, \tau')||^2$

Update ϕ via loss_ϕ

end for

end for

end for

```

# ----- Phase 3: Apply Update -----
Average grad_θ over (B * G * chunks)
Update  $\theta = \theta - \alpha * \text{grad}_\theta$ 
end while

```

实现笔记: - `compute_score( $\pi_{\text{tea}}$ , ·)`: teacher 在 noisy state 上 forward 一次拿到 velocity, 然后转为 score (与 AnyFlow 一致) - 整个 loop 直接在 StarVLA 的 RLinf 训练框架 + RL trainer 上加 OPD plugin 即可, **无需重写训练栈** - 与 GRPO 联合训练: 可以在最后 N% steps 加 GRPO outcome reward, 做 “Distill → Distill+RL” 二阶段 (参考 VLA-OPD 设计)

6 5. 实验设计

6.1 5.1 评估栈（来自用户决策）

3 个 **benchmark** + 真机补充:

Benchmark	评估范围	与 VLA-OPD 对比	资源
LIBERO (4 suite: Spatial/Object/Goal/-Long)	单臂操作	✓ 直接 head-to-head	~50 GPU-hour/method
RoboTwin 2.0 (4 dual-arm task)	双臂复杂	✓ 直接 head-to-head	~80 GPU-hour/method
SimplerEnv (WidowX 4 task) (Optional) 真机	sim2real 小 gap Sim2real demo	π _RL / STARE-VLA 用过 我们已有基础	~30 GPU-hour/method 1-2 简单任务

Backbone choices (来自 StarVLA): - 主: StarVLA-PI (π 0 风格 flow matching action expert)
- 次: StarVLA-GR00T (双系统架构) — 验证可迁移性

6.2 5.2 Baseline 矩阵

Baseline	用途
Student SFT 初始化 (1-traj or 1k-traj)	Lower bound
Full-data SFT (50 traj/task)	Data-rich upper bound
VLA-OPD (token-level)	直接对比同 OPD 思想
π _RL (Flow-SDE + PPO)	对比 RL on flow VLA
STARE-VLA (IPI pipeline)	对比 stage-shaped RL
OFP (One-Step Flow Policy, self-distill)	对比无 teacher distillation
Teacher π _tea (SimpleVLA-RL)	Performance oracle / upper bound
Ours: Flow-VLA-OPD (Distill only)	Main result
Ours: Flow-VLA-OPD + GRPO	Best result

6.3 5.3 核心实验问题（必答）

1. **Q1 (Efficacy)**: Flow-VLA-OPD 在 1-traj SFT 起点上能否达到 $\geq 85\%$ Teacher 性能?
2. **Q2 (vs RL)**: 在相同 GPU-hour 预算下, Flow-VLA-OPD 是否 $\geq \pi$ _RL 的最终 success rate?
3. **Q3 (Sample Eff)**: 达到 80% success rate 所需的 environment interaction, Flow-VLA-OPD vs π _RL/RLinf 是否 $\geq 5\times$ 加速?
4. **Q4 (vs Token-OPD)**: 同任务上 Flow-VLA-OPD vs VLA-OPD (VLA-OPD 跑在 OpenVLA-OFT, ours 跑在 π 0), continuous formulation 是否更优?
5. **Q5 (抗遗忘)**: 在 LIBERO seen-unseen tradeoff 上, Flow-VLA-OPD 是否优于 SFT 和 RL (参考 VLA-OPD Fig.3)?

6.4 5.4 关键消融研究

消融	测试什么
ACBS shortcut count (1/2/3)	shortcut decomposition 的最优 trade-off
Teacher choice (SimpleVLA-RL vs π _RL trained vs same-backbone bigger)	Teacher quality 对结果的影响
s_fake critic capacity	几乎从 free 上升到 student 同 size 的 critic 性能曲线
Distill vs Distill+GRPO	二阶段是否 worth it
Group size G (2/4/8)	方差 vs 算力 trade-off
Forward KL vs Hard CE vs Reverse KL (DMD)	复现 VLA-OPD Fig.4 的核心 ablation

6.5 5.5 真机补充实验（可选 Section 6）

按你提到“我们已有基础”：- 1-2 个简单任务（pick-place / pour）- 在 SimplerEnv 上训练，零样本/小样本 deploy 到真机 - 主要展示 sim2real 可迁移性，不作为论文主结果

7 6. 预期贡献 & 风险

7.1 6.1 预期贡献（写论文时的卖点）

1. 首个 **Flow-VLA-OPD** 框架 —— 把 OPD 从 token-level 扩展到 continuous action flow matching, 覆盖业界主力 $\pi_0/\pi_{0.5}/GR00T$ 。
2. **Action Chunk Backward Simulation (ACBS)** —— AnyFlow shortcut decomposition 在 VLA action chunk 上的适配, 与 chunk size 解耦。
3. 跳过 **log-likelihood** 困境 —— 用 DMD-style score difference 直接做 reverse-KL, 不需要 **Flow-Noise / Flow-SDE** 的 **MDP** 改造 (π_{RL} 整篇核心贡献)。
4. 完整开源 —— 集成进 StarVLA codebase, 提供 LIBERO/RoboTwin2.0/SimplerEnv 3 个 benchmark 的训练 + 评估脚本。
5. **(Stretch) 真机 case study** —— 证明在 SimplerEnv 训练的 Flow-VLA-OPD 模型可零样本 / 小样本迁移真机。

7.2 6.2 主要风险与缓解

风险	概率	影响	缓解措施
R1: DMD score difference 在低维 action (6-DoF) 上估计噪声大	中	大	(a) 增大 group size G (b) 用 σ -discretization 离散噪声水平 (c) 借鉴 SDM Policy 的二阶段优化
R2: Teacher 与 student 支持集太远, OPD 失败 (参考 Rethinking OPD 论文 phenomenology)	中	大	(a) 强制 SFT cold start (b) 选用同 backbone 的 RL-tuned teacher, 保证 mode 一致
R3: π_{RL} / STARE-VLA 6 个月后又出现升级版抢身位	高	中	(a) 立即抢 timing, 目标 2 个月内 arxiv (b) 强调 distillation vs RL 的根本范式差异
R4: ACBS 在 VLA action chunk (K=4) 上 shortcut decomposition 没有 AnyFlow 那样省	中	中	设 fallback: K=4 直接 full backward, $K \geq 8$ 才用 ACBS。这种情况下我们的卖点重心移到 score-based reward
R5: StarVLA 框架 buggy / 文档不全	中	中	(a) 第 1 周专门 setup 验证 (b) 备选 Lelsaac / OpenVLA-OFT 原仓库
R6: 真机实验失败	高	小	真机本来就是补充实验, 主结果不依赖

7.3 6.3 论文 framing 的 fallback 方案

如果主 thesis 在某个 benchmark 上失败 (比如 RoboTwin 不如 π_{RL}), fallback: - **Plan B:** 重新 frame 为 “Distillation + RL 的最佳组合” —— 用 Flow-VLA-OPD 做 warm-up, π_{RL} /STARE-VLA

做 fine-tune - **Plan C**: 缩小 scope 到 “Flow-based VLA 在数据稀缺 (1-traj) setting 下的最佳后训练” —— 此时 VLA-OPD 思路就有强 motivation

8 7. 时间线与资源

8.1 7.1 假设资源

- **GPU:** gpu-server 8×RTX 5090 (已有), featurize 6×RTX 4090 (备用)
- **Models:** $\pi_0/\pi_{0.5}$ 公开 checkpoint (HuggingFace), StarVLA-PI / StarVLA-GR00T 训练好版本 (HF)
- **Simulators:** LIBERO + RoboTwin 2.0 + SimplerEnv (StarVLA 已集成)

8.2 7.2 三个月时间线 (目标 2026-08 arxiv 投稿)

周次	里程碑	风险点
W1-2 (5 月)	StarVLA setup + LIBERO baseline 跑通; 团队熟悉 RLinf	StarVLA 上手坑
W3-4 (6 月)	实现 Flow-VLA-OPD prototype + ACBS; LIBERO-Object 上 1-traj 复现 $\geq 80\%$	DMD score noise
W5-6 (6 月)	完整 LIBERO 4 suite 跑通; 初步对比 VLA-OPD 的 distill-only 结果	Teacher 选型
W7-8 (7 月)	RoboTwin 2.0 4 task + SimplerEnv 4 task 完整结果	双臂复杂度
W9-10 (7 月)	全部消融实验 (5 个); π_{RL} / STARE-VLA / VLA-OPD 三个 baseline 完整对比	baseline 复现
W11-12 (8 月)	论文写作 + 真机补充实验 + arxiv 投稿	时间不足

8.3 7.3 关键决策点 (gating questions)

每 2 周必须回答: 1. 第 2 周末: StarVLA + 1 个 baseline (SFT) 能跑通 LIBERO 吗? (go/no-go) 2. 第 4 周末: Flow-VLA-OPD prototype 在 LIBERO-Object 上 1-traj 是否 $\geq$ Lower bound + 30%? 3. 第 8 周末: 3 个 benchmark 全部跑出结果, 主结果是否 $\geq$ VLA-OPD? 4. 第 10 周末: 消融全部 done, 是否所有核心 claim 都有数据支撑?

如果任一 gate 不过, 立即切 fallback plan B/C.

9 8. 参考资料（按引用顺序）

9.1 直接相关论文

1. **VLA-OPD**: Zhong et al. *Bridging Offline SFT and Online RL for VLA Models via On-Policy Distillation*. arXiv 2603.26666. [link](#)
2. **AnyFlow**: Gu et al. *AnyFlow: Any-Step Video Diffusion Model with On-Policy Flow Map Distillation*. arXiv 2605.13724. [link](#)
3. **π _RL**: Chen et al. *π RL: Online RL Fine-tuning for Flow-based Vision-Language-Action Models*. arXiv 2510.25889. [link](#)
4. **STARE-VLA**: Xu et al. *Progressive Stage-Aware Reinforcement for Fine-Tuning VLA Models*. arXiv 2512.05107. [link](#)
5. **DiG-Flow**: BeingBeyond. *Discrepancy-Guided Flow Matching for Robust VLA Models*. arXiv 2512.01715. [link](#)
6. **ConRFT**: Chen et al. *A Reinforced Fine-tuning Method for VLA Models via Consistency Policy*. arXiv 2502.05450. [link](#)
7. **One-Step Flow Policy**: arXiv 2603.12480. [link](#)
8. **SDM Policy**: arXiv 2412.09265. [link](#)
9. **StarVLA**: A Lego-like Codebase. arXiv 2604.05014. [link](#)
10. **DMD**: Yin et al. *One-step Diffusion with Distribution Matching Distillation*. CVPR 2024.

9.2 OPD 理论基础

11. **On-Policy Distillation** Blog: Kevin Lu, Thinking Machines Lab, 2025-10. [link](#)
12. **Rethinking OPD**: arXiv 2604.13016 (phenomenology / mechanism / recipe)
13. **MeanFlow** (AnyFlow Stage 1 来源): arXiv 2505.13447
14. **Align Your Flow** (NVIDIA, continuous-time flow map): arXiv 2506.14603

9.3 VLA Backbone & Benchmark

15. **π 0**: Black et al. arXiv 2410.24164
16. **π 0.5**: Physical Intelligence. arXiv 2504.16054
17. **GR00T N1**: NVIDIA. arXiv 2503.14734
18. **OpenVLA-OFT**: arXiv 2502.19645
19. **SimpleVLA-RL**: arXiv 2509.09674 (本工作主 teacher 候选)
20. **LIBERO**: Liu et al. NeurIPS 2023
21. **RoboTwin2.0**: arXiv 2506.18088
22. **SimplerEnv**: Li et al. 2024

9.4 已抓取本地解读

- references/05_thinking-machines_on-policy-distillation.md — OPD 原始定义
- references/02_qingkeai_rethinking-opd.md — OPD 失败模式分析
- references/06_qingkeai_seven-papers-opd-deep-dive.md — OPD 论文综述
- docs/AnyFlow-and-VLA-OPD-Technical-Review.md — AnyFlow + VLA-OPD 技术拆解（本仓库前一份 doc）

文档版本: v0.1 (2026-05-15) 状态: 提案草稿 / 等待用户评审 + 修订 所属仓库: <https://github.com/miracle-techlink/opd-papers>

【评审】请你评审反馈: 1. Novelty framing 够不够清晰? 是否有需要补强的差异化点? 2. 实验设计是否够 conservative? 还是应该增加 challenging task (如 BEHAVIOR-1K)? 3. 3 个月时间线是否激进? 需要不需要砍 scope? 4. 是否需要在第 1 周先做 prototype 验证再投入完整论文工作?